

Aula 08 – CSS Grid

1. Abertura da Aula

Sejam muito bem-vindos a mais uma aula de Desenvolvimento de Sistemas! Na nossa última aula, dominamos o Flexbox e vimos como ele é incrível para alinhar elementos em uma direção (linha ou coluna). No entanto, o mercado exige a construção de interfaces cada vez mais complexas, como painéis administrativos (dashboards), onde precisamos controlar simultaneamente as linhas e as colunas de uma página. É exatamente para resolver esse problema que existe o CSS Grid Layout, a ferramenta mais poderosa do CSS para criação de layouts bidimensionais. Dentro da nossa Situação de Aprendizagem do semestre, o "Desenvolvimento e Implantação de Soluções Digitais Inovadoras", vocês precisarão construir uma aplicação web que será apresentada para investidores anjo. Ter um painel de controle bem desenhado, estruturado e organizado é fundamental para transmitir credibilidade e facilitar a usabilidade do usuário final. Hoje, vocês aprenderão a desenhar a arquitetura visual dessas telas complexas, dando um salto gigante na qualidade do frontend da equipe.

2. Conceitos Fundamentais

2.1. O que é o CSS Grid e a definição de Tracks (Linhas e Colunas)

1. Explicação teórica

O CSS Grid Layout é um sistema de layout bidimensional para a web. Ao contrário do Flexbox, que lida predominantemente com uma dimensão por vez (ou horizontal ou vertical), o Grid foi construído para lidar com as duas dimensões (linhas e colunas) ao mesmo tempo. Ao declararmos `display: grid` em um elemento pai (o contêiner), transformamos seu contexto de formatação. A partir daí, utilizamos propriedades como `grid-template-columns` e `grid-template-rows` para definir os "tracks" (trilhas), que nada mais são do que o espaço entre duas linhas do grid. Para facilitar a criação de grids fluidos, o CSS introduziu uma unidade de medida chamada fracionária (`fr`), que representa uma fração do espaço livre disponível no contêiner. Usar `fr` elimina a necessidade de calcular porcentagens exatas ou trabalhar com larguras fixas em pixels, permitindo que o navegador faça a matemática da responsividade por você de forma extremamente otimizada e eficiente.

2. Demonstração prática

CSS

```
/* O contêiner pai que abrigará o Grid */
.grid-container {
  display: grid;
  /* Cria 3 colunas: a 1ª ocupa 1 fração, a 2ª ocupa 2 frações (o dobro), a 3ª ocupa 1 fração */
  grid-template-columns: 1fr 2fr 1fr;
  /* Cria 2 linhas com 150px de altura cada */
  grid-template-rows: 150px 150px;
  /* Adiciona um espaçamento de 10px entre as colunas e linhas */
  gap: 10px;
  background-color: #e0e0e0;
  padding: 10px;
}

/* Estilo apenas visual para os itens filhos */
.grid-item {
  background-color: #005A9C;
  color: white;
```

```
padding: 20px;
font-size: 1.2rem;
}
```

HTML

```
<div class="grid-container">
  <div class="grid-item">1</div>
  <div class="grid-item">2</div>
  <div class="grid-item">3</div>
  <div class="grid-item">4</div>
  <div class="grid-item">5</div>
  <div class="grid-item">6</div>
</div>
```

Explicação do código: Ao aplicar `grid-template-columns: 1fr 2fr 1fr`, o navegador divide o espaço horizontal em 4 partes iguais (1+2+1). A primeira coluna ganha 1 parte, a segunda ganha 2 partes (ficando mais larga) e a terceira ganha 1 parte. Os elementos filhos (`.grid-item`) fluem automaticamente para dentro dessas células na ordem em que aparecem no HTML.

3. Exercício guiado

No seu editor de código, crie um arquivo HTML com uma `div` contendo quatro caixas (`div` filhas). No CSS, transforme a `div` pai em um grid de 2 colunas e 2 linhas. Use a unidade `fr` para que as duas colunas tenham exatamente o mesmo tamanho (`1fr 1fr`).

4. Exercício de fixação

Altere o exercício anterior para que ele tenha 4 colunas em vez de 2. A primeira e a última coluna devem ter `1fr`, e as duas colunas do meio devem ter `2fr`. Adicione a propriedade `gap: 15px`; e observe o resultado.

2.2. Grid Areas (Áreas de Grade Nomeadas)

1. Explicação teórica

Uma das funcionalidades mais impressionantes e legíveis do CSS Grid é a propriedade `grid-template-areas`. Ela permite que você nomeie áreas específicas do seu layout e, em seguida, posicione os itens dentro dessas áreas usando a propriedade `grid-area` em cada elemento filho. Essa técnica torna o código CSS extremamente visual, quase como um desenho no formato "ASCII art". É uma abordagem altamente semântica, pois qualquer desenvolvedor que olhar o código do seu contêiner entenderá instantaneamente a topologia da página, sem precisar decifrar números complexos de linhas e colunas. Além disso, o uso de `grid-template-areas` facilita drasticamente a criação de layouts responsivos, já que você pode simplesmente reescrever o "desenho" das áreas dentro de uma media query para reorganizar blocos inteiros da página (como mover a barra lateral para baixo do conteúdo principal) sem tocar no HTML.

2. Demonstração prática

CSS

```
.layout-principal {
```

```

display: grid;
/* Desenha o layout visualmente com os nomes das áreas */
grid-template-areas:
  "header header header"
  "menu conteudo propaganda"
  "footer footer footer";
/* Define o tamanho das colunas para cada área nomeada acima */
grid-template-columns: 200px 1fr 150px;
/* Define a altura mínima da linha do meio */
grid-auto-rows: minmax(100px, auto);
gap: 10px;
}

/* Agora, mapeamos cada classe HTML para a sua respectiva área no desenho acima */
.cabecalho { grid-area: header; background: #333; color: white; }
.navegacao { grid-area: menu; background: #4f4f4; }
.principal { grid-area: conteudo; background: #fff; border: 1px solid #ccc; }
.anuncio { grid-area: propaganda; background: #ffcc00; }
.rodape { grid-area: footer; background: #333; color: white; }

```

3. Exercício guiado

Crie uma estrutura HTML simulando uma postagem de blog, contendo um título, uma imagem de destaque e o texto do artigo. Usando `grid-template-areas`, desenhe um layout onde o título ocupa a parte superior inteira, a imagem fica à esquerda e o texto à direita.

4. Exercício de fixação

Modifique o "desenho" no CSS do exercício guiado para que, em vez de ficarem lado a lado, a imagem fique no topo, o título fique no meio e o texto fique embaixo. Perceba como você alterou toda a estrutura da página apenas mudando palavras no CSS, sem alterar a ordem do HTML.

2.3. A grande diferença: Flexbox × Grid

1. Explicação teórica

A dúvida mais comum entre desenvolvedores iniciantes é: "Quando devo usar Flexbox e quando devo usar Grid?". A resposta está na dimensionalidade e no controle do layout. O Flexbox é "content-first" (orientado ao conteúdo) e unidimensional: ele é brilhante para pegar um grupo de itens, alinhá-los em uma linha ou coluna, e distribuir o espaço entre eles dependendo do tamanho de cada item. Já o CSS Grid é "layout-first" (orientado ao layout) e bidimensional: você define a estrutura rígida de linhas e colunas primeiro, e depois diz aos itens onde eles devem se encaixar nessa grade. Na prática profissional, eles não são concorrentes, mas sim parceiros. O padrão arquitetural de mercado é utilizar o CSS Grid para a estrutura macro da página (posicionar o cabeçalho, a barra lateral, o rodapé) e utilizar o Flexbox para a microestrutura (alinhar os links dentro do cabeçalho, ou centralizar o ícone e o texto dentro de um botão).

2. Demonstração prática

CSS

```

/* MACRO ESTRUTURA: Usando GRID para a tela principal */

```

```

.tela-dashboard {
  display: grid;
  grid-template-columns: 250px 1fr;
  grid-template-rows: 80px 1fr;
  height: 100vh;
}

/* MICRO ESTRUTURA: Usando FLEXBOX dentro de um item do Grid */
.dashboard-header {
  /* Este elemento é um filho do Grid, mas também é um contêiner Flex */
  display: flex;
  justify-content: space-between; /* Empurra os itens para as bordas */
  align-items: center; /* Centraliza verticalmente */
  background-color: white;
  padding: 0 20px;
  box-shadow: 0 2px 5px rgba(0,0,0,0.1);
}

```

3. Exercício guiado

Crie um contêiner `.grid-cards` usando `display: grid` com 3 colunas (`1fr 1fr 1fr`). Dentro dele, crie 3 cards (div). Para cada card, adicione uma imagem, um título e um botão. Transforme cada card individualmente em um `display: flex; flex-direction: column;` para alinhar os elementos internos, garantindo que o botão fique sempre no final do card usando `margin-top: auto;`.

4. Exercício de fixação

Olhe para o projeto da sua equipe. Escreva em comentários no seu arquivo CSS três exemplos de onde você utilizaria Grid (ex: layout da página de produtos) e três exemplos de onde utilizaria Flexbox (ex: botões de curtir e compartilhar lado a lado).

3. Demonstração Aplicada

Abaixo, temos um exemplo completo da construção de um layout clássico de Dashboard administrativo. Percebam como o Grid resolve o layout macro e o Flexbox cuida dos alinhamentos internos.

HTML

```

<!DOCTYPE html>
<html lang="pt-br">
<head>
  <meta charset="UTF-8">
  <title>Dashboard do Sistema</title>
  <style>
    * { margin: 0; padding: 0; box-sizing: border-box; font-family: sans-serif; }

    /* GRID: Layout Macro */
    .dashboard-container {
      display: grid;
      grid-template-areas:
        "sidebar header"

```

```
    "sidebar main";
    grid-template-columns: 260px 1fr;
    grid-template-rows: 70px 1fr;
    height: 100vh;
}
```

```
.sidebar {
    grid-area: sidebar;
    background-color: #1a1a2e;
    color: white;
    padding: 20px;
}
```

/* FLEXBOX: Alinhamento interno da sidebar */

```
.sidebar-menu {
    display: flex;
    flex-direction: column;
    gap: 15px;
    margin-top: 30px;
}
```

```
.header {
    grid-area: header;
    background-color: #ffffff;
    border-bottom: 1px solid #e0e0e0;
    padding: 0 20px;
    /* FLEXBOX: Alinhamento interno do header */
    display: flex;
    justify-content: space-between;
    align-items: center;
}
```

```
.main-content {
    grid-area: main;
    background-color: #f4f7f6;
    padding: 30px;
    overflow-y: auto;
}
```

/* GRID: Grade de estatísticas dentro do conteúdo principal */

```
.stats-grid {
    display: grid;
    grid-template-columns: repeat(3, 1fr); /* Atalho: cria 3 colunas de 1fr */
    gap: 20px;
    margin-top: 20px;
}
```

```
.stat-card {
    background: white;
    padding: 20px;
    border-radius: 8px;
```

```

    box-shadow: 0 4px 6px rgba(0,0,0,0.05);
  }
</style>
</head>
<body>
<div class="dashboard-container">
  <aside class="sidebar">
    <h2>SenaiTech</h2>
    <nav class="sidebar-menu">
      <a href="#" style="color:white; text-decoration:none;">Início</a>
      <a href="#" style="color:white; text-decoration:none;">Relatórios</a>
      <a href="#" style="color:white; text-decoration:none;">Configurações</a>
    </nav>
  </aside>

  <header class="header">
    <div>Pesquisar...</div>
    <div>Perfil do Usuário</div>
  </header>

  <main class="main-content">
    <h1>Visão Geral do Sistema</h1>
    <div class="stats-grid">
      <div class="stat-card"><h3>Usuários</h3><p>1.240</p></div>
      <div class="stat-card"><h3>Vendas</h3><p>R$ 45.000</p></div>
      <div class="stat-card"><h3>Acessos Hoje</h3><p>342</p></div>
    </div>
  </main>
</div>
</body>
</html>

```

4. Casos Reais do Mercado

- **Painéis Administrativos (SaaS / ERPs):** Empresas que desenvolvem softwares de gestão, como a SAP ou a TOTVS, utilizam exaustivamente o CSS Grid. A estrutura de manter um menu lateral fixo (sidebar) enquanto o conteúdo rola de forma independente na área principal é feita declarando `grid-template-columns` e travando a altura com `100vh`. Isso evita que o menu desapareça quando o usuário desce a página.
- **Galerias de Mídia e E-commerces:** Plataformas como o Pinterest e catálogos de lojas virtuais utilizam o CSS Grid combinando a propriedade `grid-template-columns: repeat(auto-fill, minmax(200px, 1fr));`. Essa linha de código avançada permite que o sistema crie quantas colunas couberem na tela automaticamente, redimensionando as imagens perfeitamente sem precisar usar media queries.

5. FAQ – Dúvidas Reais de Alunos

1. Posso colocar um Grid dentro de outro Grid?

R: Sim, absolutamente! Assim como no HTML colocamos divs dentro de divs, você pode ter um

item do Grid (uma célula) que também recebe `display: grid` para estruturar seu conteúdo interno de forma ainda mais detalhada.

2. **O que é a função `repeat()` que vi no exemplo aplicado?**

R: É um atalho do CSS Grid. Em vez de digitar `1fr 1fr 1fr 1fr`, você pode digitar `repeat(4, 1fr)`. Torna o código muito mais limpo e profissional.

3. **O CSS Grid funciona em navegadores antigos?**

R: Hoje em dia, o suporte ao CSS Grid é superior a 96% nos navegadores globais. Para versões muito antigas do Internet Explorer, ele pode não funcionar corretamente, mas para o mercado atual, é o padrão absoluto e seguro de usar.

4. **A propriedade `gap` substitui a margem (`margin`)?**

R: No contexto do contêiner, sim! A propriedade `gap` adiciona espaçamento apenas *entre* os elementos, não criando espaço extra nas bordas externas. Isso resolve o problema antigo de margens que "quebravam" o layout da linha.

5. **Quando eu uso `grid-template-areas`, preciso definir o tamanho de cada área lá mesmo?**

R: Não. O `grid-template-areas` apenas desenha o mapa conceitual. Os tamanhos exatos dessas áreas são controlados separadamente pelas propriedades `grid-template-columns` e `grid-template-rows`.

6. Desafio Prático

Contexto: Vocês estão desenvolvendo a solução digital inovadora para a Situação de Aprendizagem. O sistema web precisa de uma interface gráfica administrativa onde os gestores da empresa verão os dados gerados.

Requisitos:

1. Cada equipe deverá desenhar e codificar o **Dashboard** principal da sua aplicação (ou a tela mais complexa do sistema que envolva múltiplas colunas/áreas).
2. O layout principal da tela (sidebar, header, conteúdo central, etc.) **deve obrigatoriamente** ser construído utilizando CSS Grid Layout (de preferência usando `grid-template-areas`).
3. O conteúdo interno dos componentes (como alinhar textos, ícones ou botões dentro dos painéis/cards) **deve** utilizar Flexbox.
4. Ao final do código, a equipe deve inserir comentários justificando a escolha das tecnologias: "Usamos Grid aqui porque..." e "Usamos Flexbox aqui porque...".

Resultado Esperado: Um arquivo HTML estruturado e um CSS robusto contendo o painel de controle do sistema da equipe.

Sugestão de Solução: Reúnam-se e esboquem o layout no papel ou no quadro antes. Desenhem a grade, identifiquem quantas linhas e colunas serão necessárias e deem nomes claros para cada área. Só depois abram o editor de código.

7. Conexão com a Situação de Aprendizagem

Este painel administrativo (Dashboard) que vocês estão estruturando hoje é o coração visual da aplicação que os investidores anjo vão testar. Nas próximas etapas da unidade curricular de "Backend: PHP, Banco de Dados, CRUD", os cards estáticos que vocês construíram hoje passarão a exibir dados dinâmicos. Por exemplo, em vez de um texto digitado no HTML dizendo "342 acessos", o sistema fará uma requisição (query) no banco de dados MySQL via PHP, calculará os números reais da plataforma e injetará esse valor dentro do grid de vocês. Ter uma estrutura sólida no CSS garantirá que, mesmo que o banco de

dados retorne milhares de registros ou textos longos, a interface não vai "quebrar" ou perder o alinhamento.

8. Preparação para a Próxima Aula

Hoje nós solidificamos a nossa estrutura de layout estática e dominamos o HTML e CSS para posicionamento de elementos em tela. Mas, para a nossa solução digital estar verdadeiramente completa, ela não pode ser estática. Na nossa próxima aula, avançaremos para a Unidade 3: **Interatividade e Lógica Client-Side (JavaScript)**. Vocês aprenderão a dar "vida" a esse Dashboard. Ensinaremos como fazer com que botões executem funções, como abrir e fechar a barra lateral com um clique e como manipular elementos na tela do usuário em tempo real sem recarregar a página. Guardem o layout de hoje com carinho, pois ele será a base onde injetaremos a inteligência do JavaScript!